

10041 - Vito's Family

Name: Shashwati Gavhale

Roll: 352

PRN: 23046491245121

Date of Uploading: 15/04/2026

Problem Description

- You are given **two strings** **a** and **b**.
- Both strings contain only **lowercase letters (a-z)**.
- You need to find the **longest possible string x** such that:
 - some **permutation of x** is a subsequence of **a**
 - some **permutation of x** is a subsequence of **b**
- Since permutation is allowed, **order does not matter**
- This means we only need the **common characters** present in both strings
- Each character in **x** should appear **minimum frequency** times from both strings
- If multiple answers are possible, print the **alphabetically smallest one**
- The output must be printed for **every pair of input lines**

Proposed Solution

Read the input **two lines at a time**

- first line → string **a**
- second line → string **b**

Create **two frequency arrays** of size **26**

- one for string **a**
- one for string **b**

Count how many times each character appears in both strings

- `freqA[ch - 'a']++`
- `freqB[ch - 'a']++`

Traverse all characters from **a to z**

For each character, find the **minimum frequency** in both strings

`common = min(freqA[i], freqB[i])`

Add that character to the result exactly **common** times

Since characters are added from **a to z**,

the result is automatically the **alphabetically smallest**

Print the final string for every pair of input lines

Uploaded Code

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (sc.hasNextLine()) {
            String a = sc.nextLine();

            if (!sc.hasNextLine()) break;
            String b = sc.nextLine();

            int[] freqA = new int[26];
            int[] freqB = new int[26];

            // Count frequency of characters in first string
            for (char ch : a.toCharArray()) {
                freqA[ch - 'a']++;
            }

            // Count frequency of characters in second string
            for (char ch : b.toCharArray()) {
                freqB[ch - 'a']++;
            }

            StringBuilder result = new StringBuilder();

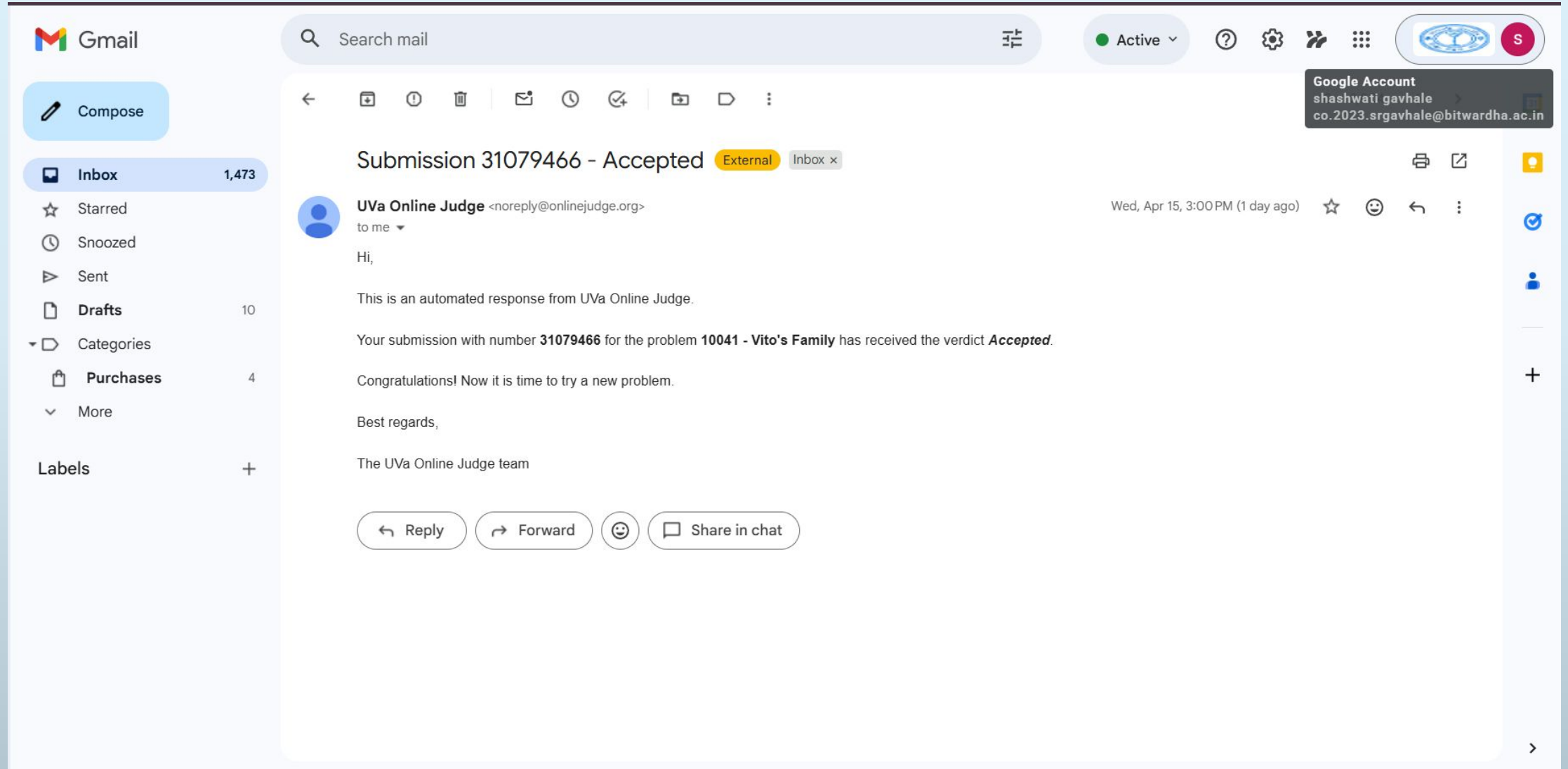
            // Build answer in alphabetical order
            for (int i = 0; i < 26; i++) {
                int common = Math.min(freqA[i], freqB[i]);

                for (int j = 0; j < common; j++) {
                    result.append((char) (i + 'a'));
                }
            }

            System.out.println(result.toString());

            sc.close();
        }
    }
}
```

Screenshot of accepted solution



120 - Stacks of FlapJacks

Name: Shashwati Gavhale

Roll: 352

PRN: 23046491245121

Date of Uploading: 16/04/2026

Proposed Description

- You are given a rectangular grid of letters representing a word search puzzle
- A list of words is provided that must be searched inside the grid
- Each word can appear in any of the **8 possible directions** (horizontal, vertical, diagonal)
- The search is **case-insensitive** (uppercase and lowercase are treated the same)
- Each word is guaranteed to appear at least once in the grid
- You must locate the **first occurrence** of each word
- The output should be the **starting position (row and column)** of the word
- Positions are reported using **1-based indexing**
- If multiple matches exist, choose the one with the **smallest row, then smallest column**

Proposed Solution

Compare each line with the key sentence.

Check:

- same length
- spaces at same positions
- one-to-one character mapping

If valid, create mapping using **HashMap**.

Use mapping to decrypt all lines.

Otherwise print **No solution.**

Uploaded Code

```
import java.io.*;
import java.util.*;

public class Main {
    static final String KEY = "the quick brown fox jumps over the lazy dog";

    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        int t = Integer.parseInt(br.readLine());
        br.readLine(); // blank line after test cases

        while (t-- > 0) {
            List<String> lines = new ArrayList<>();
            String line;

            // Read one test case until blank line or EOF
            while ((line = br.readLine()) != null && !line.isEmpty()) {
                lines.add(line);
            }

            Map<Character, Character> map = null;

            // Find the line that can map to the key sentence
            for (String s : lines) {
                map = getMapping(s, KEY);
                if (map != null) break;
            }

            if (map == null) {
                System.out.println("No solution.");
            } else {
                for (String s : lines) {
                    StringBuilder decrypted = new StringBuilder();
                    for (char c : s.toCharArray()) {
                        if (c == '*') {
                            decrypted.append(' ');
                        } else {
                            decrypted.append(map.get(c));
                        }
                    }
                    System.out.println(decrypted);
                }
            }

            if (t > 0) System.out.println();
        }

        static Map<Character, Character> getMapping(String encrypted, String target) {
            if (encrypted.length() != target.length()) return null;

            Map<Character, Character> map = new HashMap<>();
            Map<Character, Character> reverse = new HashMap<>();

            for (int i = 0; i < encrypted.length(); i++) {
                char e = encrypted.charAt(i);
                char t = target.charAt(i);

                if (e == '*' && t == '*') continue;
                if (e == '*' || t == '*') return null;

                if (map.containsKey(e)) {
                    if (map.get(e) != t) return null;
                } else {
                    map.put(e, t);
                }

                if (reverse.containsKey(t)) {
                    if (reverse.get(t) != e) return null;
                } else {
                    reverse.put(t, e);
                }
            }

            return map;
        }
    }
}
```

Screenshot of accepted solution

